

Documentation du Protocole de Communication

(esp32_sensor)

Ce document décrit en détail les protocoles d'échange et de communication exposés par le micrologiciel **esp32_sensor** (projet **mir**) à travers ses différents canaux d'E/S : Liaison série (UART), TCP texte (commandes), TCP binaire (flux de données capteur), UDP (découverte réseau) et BLE (provisioning WiFi).

1. Vue d'ensemble des Transports & Canaux

Le système utilise 5 canaux de communication distincts :

Transport	Port	Type / Format	Usage principal	Restrictif / Spécificité
Liaison Série	— (UART)	Texte ASCII (\n)	Configuration initiale, diagnostic, gestion WiFi	Accès complet (Credentials WiFi autorisés)
TCP Commandes	5000	Texte ASCII (\n)	Configuration à distance, interrogation	Restreint (Credentials WiFi interdits)
TCP Données	5001	Binaire (Little-Endian)	Stream haute fréquence des données capteur	Flux binaire continu orienté trames
UDP Découverte	5679 (In) 5678 (Out)	Texte ASCII / JSON	Auto-découverte des modules sur le réseau	Détection dynamique de l'IP du module
BLE Provisioning	— (GATT)	GATT (Write / Read+Notify)	Provisioning WiFi sans fil au boot	Accès complet (SSID/PWD autorisés, fenêtre 60 s)

Paramètres Série

- **Baudrate** : **115200** baud
- **Format** : 8 bits de données, pas de parité, 1 bit de stop (8N1)
- **Délimiteur de ligne** : **\n** (Line Feed, ASCII **0x0A**)

2. Protocole de Commandes Texte (Série & TCP Port 5000)

Le protocole de commande est un protocole ASCII synchrone basé sur des lignes terminées par **\n**. Il est identique sur la liaison Série et le serveur TCP port 5000, à l'exception de la politique de sécurité restreignant les identifiants WiFi sur TCP.

2.1. Format des Requêtes

```
<commande> <cle1>=<valeur1>;<cle2>=<valeur2>\n
```

- **Nom de la commande** : Premier mot (séparé des arguments par un espace).
- **Arguments** : Paires **clé=valeur** séparées par des points-virgules ;.
- **Limites système** :
 - Taille maximale du buffer d'entrée : **1000 octets** (**CMD_BUFFER_SIZE**).
 - Nombre maximal d'arguments par commande : **10** (**CMD_MAX_ARGS**).

2.2. Format des Réponses

Toute réponse est préfixée par le caractère # suivi du nom de la commande réceptrice :

```
#<commande> <cle1>=<valeur1>;<cle2>=<valeur2>\n
```

- **En cas de succès d'une commande d'action** :

```
#<commande> status=success\n
```

- **En cas d'erreur** :

```
#<commande> status=error;error=<motif_erreur>\n
```

Codes d'erreur standards :

- **unknown_command** : Commande non reconnue par le système.
- **serial_only_command** : Tentative d'exécution d'une commande réservée à la liaison série depuis un socket TCP.

2.3. Répertoire des Commandes

A. Commandes de Données (Série et TCP)

get_data

Demande la dernière mesure du capteur au format texte ASCII.

- **Requête** : **get_data\n**
- **Réponse** : **#get_data <valeurs_ASCII>\n**
- **Format du payload selon le capteur activé (sensor_type)** :
 - **esp_simulation / simulation** :

```
#get_data <timestamp_us>;<omega>;<theta>
```

(Exemple: `#get_data 12345678;0.500;-0.250`)

- **esp_wheather / wheather :**

```
#get_data <timestamp_us>;<temperature_C>;  
<humidité_relative_%_entier>
```

(Exemple: `#get_data 12345678;23.5;45`)

- **esp_lidar / lidar :**

```
#get_data lidar: use TCP data stream on port 5001
```

B. Configuration Générale (Série et TCP)

get_configuration

Lit les paramètres de configuration du système (hors identifiants WiFi sensibles).

- **Requête :** `get_configuration\n`
- **Réponse :**

```
#get_configuration sensor_type=<type>;sensor_id=<id>;ap1_ip=  
<ip>;ap2_ip=<ip>;wifi_timeout=<sec>;loop_period=<ms>;current_ip=<ip>\n
```

set_configuration

Met à jour un ou plusieurs paramètres système et les sauvegarde en mémoire non volatile (NVS namespace `sensor_cfg`).

- **Requête :** `set_configuration <cle>=<valeur>;...\n`
- **Champs modifiables :**

Clé	Type / Format	Description / Contraintes
<code>sensor_type</code>	Chaîne	Type de capteur (<code>simulation</code> , <code>lidar</code> , <code>wheather</code>).
<code>sensor_id</code>	Chaîne (≤ 31 chars)	Identifiant unique du nœud.
<code>ap1_ip</code>	IPv4 ou <code>auto</code>	Adresse IP statique pour l'AP 1 (<code>auto</code> = DHCP).
<code>ap2_ip</code>	IPv4 ou <code>auto</code>	Adresse IP statique pour l'AP 2 (<code>auto</code> = DHCP).

Clé	Type / Format	Description / Contraintes
wifi_timeout	Entier (≥ 2)	Délai d'attente de connexion WiFi en secondes (défaut : 5s).
loop_period	Entier ($10 \leq x < 1000$)	Période d'exécution de la boucle principale en ms (défaut : 25ms).

- Réponse :

```
#set_configuration status=success\n
```

C. Configuration WiFi & Sécurité (Liaison Série et BLE UNIQUEMENT)

Pour éviter que des identifiants réseau ne transitent en clair sur le réseau WiFi/IP, les commandes ci-dessous sont **strictement rejetées lorsqu'elles sont reçues via TCP**. Les identifiants peuvent être configurés via la **liaison série** (commandes ci-dessous) ou via le **provisioning BLE** (voir §5 : écriture GATT SSID/PWD, stockée comme `ap1_ssid/ap1_pwd` puis reboot).

get_ap_configuration

- Port autorisé : Série uniquement
- Requête : `get_ap_configuration\n`
- Réponse sur Série :

```
#get_ap_configuration ap1_ssid=<ssid1>;ap1_pwd=<pwd1>;ap2_ssid=<ssid2>;ap2_pwd=<pwd2>\n
```

- Réponse si exécutée via TCP :

```
#get_ap_configuration status=error;error=serial_only_command\n
```

set_ap_configuration

- Port autorisé : Série uniquement
- Requête : `set_ap_configuration ap1_ssid=<ssid1>;ap1_pwd=<pwd1>;ap2_ssid=<ssid2>;ap2_pwd=<pwd2>\n`
- Réponse sur Série :

```
#set_ap_configuration status=success\n
```

- Réponse si exécutée via TCP :

```
#set_ap_configuration status=error;error=serial_only_command\n
```

D. Système (Série et TCP)

reboot

Provoque le redémarrage logiciel de l'ESP32.

- Requête : `reboot\n`
- Réponse : `#reboot status=success\n` (envoyée 100 ms avant la réinitialisation matérielle).

3. Protocole TCP Flux de Données Binaires (TCP Port 5001)

Le port TCP `5001` est dédié au streaming haute fréquence et bas niveau des trames de données capteur. Toutes les valeurs multi-octets sont codées en **Little-Endian**.

3.1. Structure d'une Trame Binaire

Chaque trame transmise sur la socket binaire commence par un **en-tête fixe de 14 octets** suivi du **payload de données** dépendant du capteur activé.



Détail de l'En-tête Fixe (14 octets) :

Octets	Type	Champ	Description / Valeur
0	uint8_t	MAGIC_0	Octet de synchronisation 1 : 0xA5

Octets	Type	Champ	Description / Valeur
1	uint8_t	MAGIC_1	Octet de synchronisation 2 : 0x5A
2..5	uint32_t	sequence	Compteur de trames incrémenté à chaque envoi (0, 1, 2, ...).
6..13	uint64_t	timestamp	Temps système écoulé depuis le démarrage en microsecondes (<code>esp_timer_get_time()</code>).

3.2. Formats du Payload selon le Capteur

A. Capteur Simulation (**SensorSimulation**)

- **Taille du payload** : 10 octets (`data_len` + `omega` + `theta`)
- **Taille totale de la trame binaire** : 24 octets (14 + 10)

Octets	Type	Champ	Description
14..15	uint16_t	data_len	Nombre d'éléments / longueur symbolique (Valeur fixe = 2)
16..19	float (IEEE 754)	omega	Vitesse angulaire simulée (rad/s)
20..23	float (IEEE 754)	theta	Angle simulé (rad)

B. Capteur Météo (**SensorWeather** - AHTX0)

- **Taille du payload** : 10 octets (`data_len` + `temperature` + `rel_humidity`)
- **Taille totale de la trame binaire** : 24 octets (14 + 10)

Octets	Type	Champ	Description
14..15	uint16_t	data_len	Longueur symbolique (Valeur fixe = 4)
16..19	float (IEEE 754)	temperature	Température mesurée en °C
20..23	float (IEEE 754)	rel_humidity	Humidité relative mesurée en %

C. Capteur LIDAR (**SensorLidar** - RPLIDAR C1)

- **Taille du payload** : $2 + (N \times 5)$ octets (où N est le nombre de points dans le scan, $N \leq 500$)
- **Taille totale de la trame binaire** : $16 + (N \times 5)$ octets (14 + 2 + $N \times 5$)

Octets	Type	Champ	Description
14..15	uint16_t	count	Nombre de points de mesure N contenus dans la trame.
16 .. 15+ (N*5)	ScanPoint[N]	points	Tableau de N structures compactées ScanPoint (5 octets / point).

Structure d'un **ScanPoint** (`__attribute__((packed))`, 5 octets) :

Décalage relatif	Type	Champ	Description / Unité
+0	uint8_t	quality	Qualité / intensité de réflexion du signal (0 à 255).
+1..+2	uint16_t	angle	Angle mesuré en dixièmes de degré (0 à 3599 pour 0, 0° ... 359, 9°).
+3..+4	uint16_t	distance	Distance mesurée en centimètres (0 = mesure invalide/hors portée).

4. Protocole de Découverte Network UDP (Port 5679 / 5678)

L'ESP32 écoute sur le réseau local pour répondre automatiquement aux requêtes d'auto-découverte émises par les clients (ex. applications hôtes, robots, interfaces de contrôle).



4.1. Requête de Découverte (Client → ESP32)

- **Port de destination ESP32** : UDP 5679
- **Contenu du datagramme** : Chaîne ASCII exacte "mir_discover_request" (les fin de lignes \r ou \n sont ignorées).

4.2. Réponse de Découverte (ESP32 → Client)

- **Port de destination Client** : UDP 5678 (envoyé directement à l'IP émettrice de la requête UDP).
- **Format du payload** : Objet JSON au format ASCII texte.

Exemple de payload JSON retourné :

```
{
  "id": "esp_0",
  "type": "esp_lidar",
  "ip": "192.168.1.42"
}
```

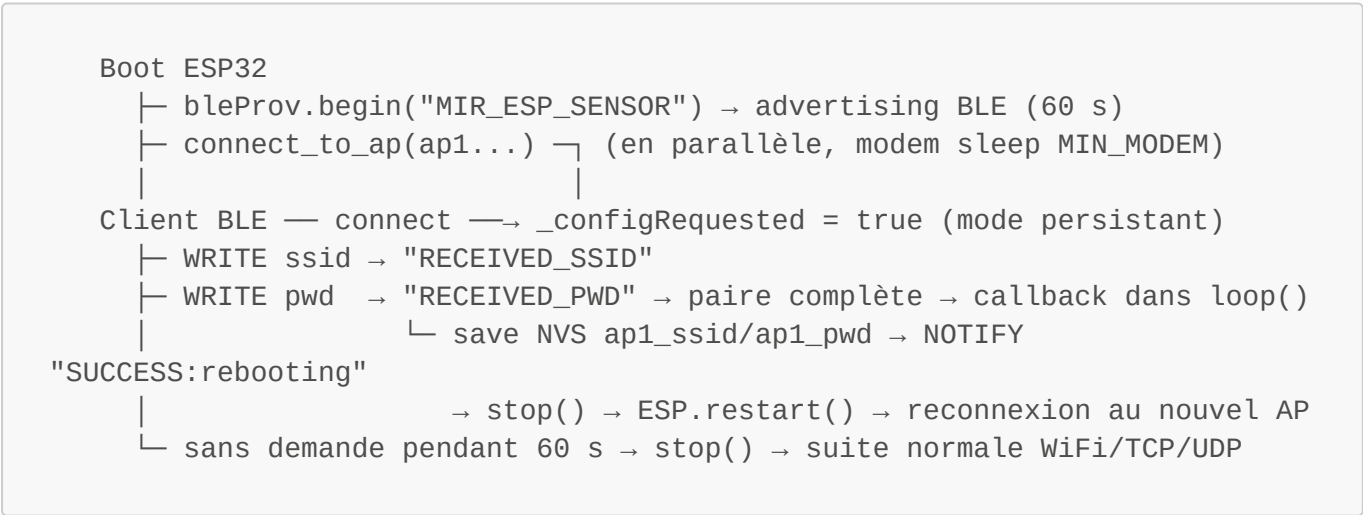
5. Provisioning WiFi via BLE (GATT)

Le provisioning BLE permet de configurer le WiFi **sans liaison série**, depuis un client BLE (ex. **nRF Connect**, script Python **bleak**). Il est implémenté par la classe **BleProvisioning** (`include/ble_provisioning.h`, `src/ble_provisioning.cpp`, stack **Bluedroid BLEDevice**) et câblé dans `src/main.cpp` (`bleProv`, `on_ble_provisioning_credentials()`).

Contrainte build (`platformio.ini`) : Bluedroid augmente fortement la taille du firmware. La partition par défaut (2 × 1,25 Mo APP) ne suffit plus, le projet impose `board_build.partitions = no_ota.csv` (1 × 2 Mo APP, pas d'OTA).

5.1. Fenêtre d'activité et cycle de vie

- **Démarrage** : `bleProv.begin("MIR_ESP_SENSOR")` est appelé dans `setup()` avant `connect_to_ap()`, pour fonctionner **en parallèle** de la connexion WiFi STA en cours. Le nom BLE est tronqué à **28 caractères** (`BLE_PROV_NAME_MAX_LEN`).
- **Fenêtre par défaut** : `DEFAULT_TIMEOUT_MS = 60 000 ms` (60 s après le boot). `bleProv.handle()` doit être appelé dans `loop()` : sans aucune demande pendant 60 s, log **BLE provisioning : aucune demande reçue pendant 60 s, arrêt.** puis `stop()`.
- **Mode configuration persistant** : dès qu'une **connexion BLE** (`handleConnect()`) ou une **écriture SSID/PWD** (`handleWrite()`) est reçue pendant la fenêtre, `_configRequested = true` et le BLE **reste actif sans limite de durée** (`handle()` retourne immédiatement). Après déconnexion, l'advertising est relancé (`handleDisconnect()` → `BLEDevice::startAdvertising()`).
- **Arrêt** : `stop()` coupe l'advertising, `BLEDevice::deinit(true)`, log **BLE provisioning arrêté.** Sans effet si inactif.
- **Coexistence WiFi + BLE** : tant que le BLE est actif, `WiFi.setSleep(WIFI_PS_MIN_MODEM)` est obligatoire (appliqué dans `begin()` et dans `connect_to_ap()` via `bleProv.isActive()`), sinon le driver WiFi aborte (**Should enable WiFi modem sleep when both WiFi and Bluetooth are enabled**). Après `stop()`, repassage en `WIFI_PS_NONE` (pleine performance WiFi).



5.2. Service et caractéristiques GATT

Un service + 3 caractéristiques, advertising avec `addServiceUUID()` + `setScanResponse(true)` :

Rôle	UUID	Propriétés	Description / Contraintes
Service	e0f0c9a0-4d1a-4e8b-9f2c-	—	Service unique de provisioning (annoncé en advertising).

Rôle	UUID	Propriétés	Description / Contraintes
	abcdef123401		
SSID	e0f0c9a0-4d1a-4e8b-9f2c-abcdef123402	WRITE	SSID du point d'accès. <code>trim()</code> appliqué, tronqué à 32 car. (<code>BLE_PROV_SSID_MAX_LEN</code>). Vide = ignoré (paire incomplète).
PWD	e0f0c9a0-4d1a-4e8b-9f2c-abcdef123403	WRITE	Mot de passe. <code>trim()</code> appliqué, tronqué à 64 car. (<code>BLE_PROV_PWD_MAX_LEN</code>). Vide accepté = réseau ouvert , mais la caractéristique doit être écrite au moins une fois (même vide, flag <code>_pwdWritten</code>).
STATUS	e0f0c9a0-4d1a-4e8b-9f2c-abcdef123404	READ NOTIFY (+ descripteur BLE2902 / CCCD)	Statut texte ASCII, valeur initiale <code>WAIT_CREDENTIALS</code> . À lire après connexion et à surveiller via notifications (activer le CCCD <code>0x2902</code>).

Notes :

- **Aucun appairage / chiffrement BLE** n'est configuré dans le code : SSID/PWD transitent en clair sur la radio locale. Réservé au provisioning de proximité, comme la liaison série.
- L'ordre d'écriture **SSID/PWD est quelconque**. Seul le contenu est journalisé sur série par sa **longueur** (`SSID reçu (N car.)`, `mot de passe reçu (N car.)`), jamais en clair.
- Les callbacks GATT s'exécutent dans le **contexte de la tâche BT** : ils ne font que lever des drapeaux (`_configRequested`, `_credentialsPending`). Le callback applicatif `onCredentials()` est exécuté dans `handle()` (**contexte `loop()`**), **après** l'envoi de la réponse ATT d'écriture. Sans cela le client recevrait une erreur GATT alors que la sauvegarde a réussi.

5.3. Caractéristique STATUS : valeurs notifiées

Envoyées par `notifyStatus()` (log série `BLE provisioning : <msg> + setValue() + notify()` si actif) :

Valeur	Émise quand
<code>WAIT_CREDENTIALS</code>	Valeur initiale à <code>begin()</code> ; renvoyée à chaque connexion BLE si aucun SSID reçu.
<code>RECEIVED_SSID</code>	Écriture SSID reçue ; renvoyée à la connexion si un SSID est déjà mémorisé.
<code>RECEIVED_PWD</code>	Écriture PWD reçue (même vide).
<code>SUCCESS:rebooting</code>	Paire complète traitée par <code>on_ble_provisioning_credentials()</code> juste avant <code>stop()</code> + <code>ESP.restart()</code> .

5.4. Traitement applicatif (`main.cpp`)

`on_ble_provisioning_credentials(ssid, pwd)` équivaut à `set_ap_configuration ap1_ssid=...; ap1_pwd=...` :

1. Stocke `ssid` → `ap1_ssid` (via `set_str_value`, vide ignoré) et `pwd` → `ap1_pwd` (copie `strncpy`, vide accepté pour réseau ouvert).
2. `save_configuration()` en NVS (namespace `sensor_cfg`).
3. Log `BLE provisioning : identifiants sauvegardés (ap1_ssid=<ssid>)`.
Redémarrage pour reconnexion....
4. `notifyStatus("SUCCESS:rebooting"), delay(500), bleProv.stop(), delay(200), ESP.restart()` → reconnexion sur le nouvel AP au boot suivant.

Seul le slot **AP1** est provisionné par BLE. Le slot AP2 reste configurable par liaison série.

5.5. Exemple de session (avec `nRF Connect` / `bleak`)

1. Au boot, repérer l'advertising `MIR_ESP_SENSOR` (service `...123401`) dans les 60 s (ou se connecter pour figer la fenêtre en mode persistant).
2. Se connecter, activer les notifications sur STATUS (`...123404`, CCCD `0x2902`) → lire `WAIT_CREDENTIALS`.
3. Écrire SSID sur `...123402` (ex. `MonReseau`) → notification `RECEIVED_SSID`.
4. Écrire PWD sur `...123403` (ex. `Secret123`, ou valeur vide pour réseau ouvert) → notification `RECEIVED_PWD`.
5. Attendre la notification `SUCCESS:rebooting` : l'ESP32 sauvegarde en NVS, coupe le BLE et reboote sur le nouvel AP. Si seul le SSID est écrit sans jamais écrire PWD, rien ne se passe (paire incomplète).

6. Résumé des Séquences et Exemples d'Échanges

Exemple 1 : Lecture de la configuration via TCP (Port 5000)

Client → ESP32 (Port 5000) :

```
get_configuration\n
```

ESP32 → Client :

```
#get_configuration
sensor_type=lidar;sensor_id=esp_0;ap1_ip=192.168.1.42;ap2_ip=auto;wifi_time
out=5;loop_period=25;current_ip=192.168.1.42\n
```

Exemple 2 : Tentative de modification des mots de passe WiFi via TCP (Port 5000)

Client → ESP32 (Port 5000) :

```
set_ap_configuration ap1_ssid=MonReseau;ap1_pwd=Secret123\n
```

ESP32 → Client (Refusé par sécurité) :

```
#set_ap_configuration status=error;error=serial_only_command\n
```

Exemple 3 : Définition des identifiants WiFi via Liaison Série (UART)

Terminal → ESP32 (Série) :

```
set_ap_configuration ap1_ssid=MonReseau;ap1_pwd=Secret123\n
```

ESP32 → Terminal :

```
#set_ap_configuration status=success\n
```

Exemple 4 : Provisioning WiFi via BLE (GATT)

Client BLE → ESP32 (dans les 60 s après boot, nom **MIR_ESP_SENSOR**) :

```
CONNECT + activer NOTIFY sur ...123404 → READ = "WAIT_CREDENTIALS"  
WRITE ...123402 = "MonReseau" → NOTIFY "RECEIVED_SSID"  
WRITE ...123403 = "Secret123" → NOTIFY "RECEIVED_PWD"
```

ESP32 → Client BLE :

```
NOTIFY "SUCCESS:rebooting" puis reboot (connexion au nouvel AP1)
```

*Document généré automatiquement d'après l'analyse du code source du projet **esp32_sensor**.*